Implementacion de clases en C++
ChessAI

Samuel Thomas Vásquez Gerena

1. Implementación de la Clase `Position`

La clase `Position` gestiona las coordenadas (fila y columna) del tablero. Siguiendo las mejores prácticas de C++, su arquitectura se divide en un archivo de cabecera (`.hpp` o contrato) y uno de implementación (`.cpp` o lógica).

Para garantizar tanto la seguridad estructural como el rendimiento computacional, la clase aplica los siguientes principios:

- **Encapsulamiento e Inicialización:** Las variables `row` y `col` son estrictamente privadas. Se inicializan de forma eficiente mediante listas de inicialización en el constructor y su acceso externo está restringido a métodos *getters*.
- **Inmutabilidad (`const`):** Todos los métodos de lectura están marcados como `const`, garantizando a nivel de compilador que consultar una posición no alterará su estado interno.
- **Paso por Referencia (`const Position&`):** Para evaluar la igualdad entre casillas, el sistema evita la copia de objetos en memoria, accediendo directamente a la dirección de memoria de forma protegida.
- **Sobrecarga del Operador (==):** Se redefinió el operador de igualdad para facilitar comparaciones directas (ej. `posA == posB`). Internamente, esto se apoya en el puntero explícito `this->`. Esta optimización reduce la sobrecarga de llamadas a funciones, lo cual es fundamental para que el algoritmo Minimax evalúe nodos de forma eficiente.

2. Implementación de la Clase `Move`

La clase `Move` actúa como un Objeto de Transferencia de Datos (DTO) que encapsula la información necesaria para ejecutar y revertir un turno dentro del motor de ajedrez. Su diseño opera como un gestor estricto de estado, fundamental para soportar la evaluación de árboles de juego en el algoritmo Minimax.

Para garantizar seguridad de memoria y rendimiento durante la simulación, la clase aplica los siguientes principios:

- **Gestión Segura de Memoria (Prevención de *Dangling Pointers*):** En lugar de utilizar punteros crudos (`Piece*`) para referenciar piezas capturadas, la clase emplea `std::unique_ptr<Piece>`. Esto asegura que, durante una captura, la pieza enemiga no se libere permanentemente de la memoria. La propiedad temporal se transfiere al objeto `Move`, preservando el dato para la operación de reversión (*undo*).

- **Encapsulamiento de la Operación Inversa:** La clase implementa los métodos `setCapturedPiece(std::unique_ptr<Piece>piece)` y `releaseCapturedPiece()`. Esto permite que el tablero delegue la propiedad de la pieza capturada al objeto `Move` y la recupere de forma determinista al revertir la jugada.

- **Alta Cohesión del Movimiento:** Cada objeto `Move` mantiene su propio contexto histórico. Al almacenar internamente la pieza capturada, se evita que clases externas (`Game` o `Board`) deban gestionar estructuras de datos auxiliares, simplificando la lógica de las llamadas recursivas del motor de IA.

- **Soporte para la Simulación del Motor IA:** Al contener toda la información transaccional, el algoritmo Minimax puede explorar y revertir estados del tablero sin incurrir en clonaciones de objetos complejos ni en asignaciones dinámicas innecesarias.

3. Implementación de la Clase `Board`

La clase `Board` representa la capa de persistencia espacial del motor. Su responsabilidad exclusiva es administrar la matriz bidimensional `grid[8][8]` de punteros inteligentes. El tablero es agnóstico a las reglas del ajedrez (turnos, jaques, colores); opera bajo la premisa de que las validaciones lógicas ya fueron procesadas por capas superiores.

Para optimizar la exploración profunda de la IA sin saturar la memoria principal, se emplea el patrón de **Operación Inversa (*Inverse Operation*)** en lugar de clonar el estado del tablero en cada nodo evaluado.

- **Paso por Referencia (`Move& m`):** Los métodos de mutación reciben un objeto `Move` por referencia modificable. Esto permite que `Board` inyecte información de estado (como la pieza capturada) directamente en el objeto sin duplicar datos.

- **Ejecución Física del Movimiento (`movePiece`):** El método `movePiece(Move& m)` ejecuta el movimiento trasladando la propiedad mediante `std::move`. Si la casilla de destino contiene un puntero válido (captura), transfiere ese `unique_ptr` al objeto `Move` mediante `m.setCapturedPiece()`, y posteriormente reubica la pieza atacante.

- **Operación Inversa (`undoPiece`):** El método `undoPiece(Move& m)` revierte la mutación. La pieza atacante retorna a su origen. Si `m.isCapture()` es verdadero, el tablero solicita la liberación del `unique_ptr` para restaurar la pieza capturada en la matriz.

- **Exploración Eficiente del Árbol de Juego:** Este diseño transaccional permite al algoritmo Minimax avanzar y retroceder en el árbol de estados minimizando el consumo de CPU y evitando operaciones costosas en el *heap* (`new` / `delete`).

4. Implementación de la Clase `Piece`

La clase `Piece` funciona como la clase base abstracta del motor. Define la interfaz común que cada pieza específica debe implementar, promoviendo el polimorfismo estructurado.

Para garantizar un diseño escalable y seguro a nivel de memoria, la clase aplica las siguientes directrices técnicas:

- **Abstracción Pura (`= 0`):** El método `getPossibleMoves` es virtual puro, impidiendo la instanciación directa de `Piece` y obligando a las clases derivadas a proveer su propia lógica de cálculo de trayectorias.
- **Seguridad de Tipos (`enum class`):** El color de la pieza se define mediante un enumerador fuertemente tipado (`Color::WHITE`, `Color::BLACK`). Esto previene errores lógicos derivados del uso de variables enteras estándar.
- **Herencia Controlada (`protected`):** El atributo de color posee un modificador de acceso protegido, ocultándolo de componentes externos mientras permite el acceso directo desde las clases derivadas.
- **Gestión de Memoria Segura (`virtual P̃iece`):** Se declara un destructor virtual explícito, una práctica estándar en C++ al emplear polimorfismo. Esto asegura que la liberación de una instancia derivada a través de un puntero base invoque la cadena de destructores correcta, previniendo fugas de memoria (*Memory Leaks*).
- **Valoración Relativa (`getValue`):** El método virtual puro obliga a definir el peso estandarizado de cada pieza. Esto provee la base heurística para la evaluación de material en algoritmos de inteligencia artificial.

4.1. Implementación de la Subclase `Pawn` (Peón)

La clase `Pawn` presenta un modelo de movimiento asimétrico y unidireccional, separando lógicamente el avance de la captura.

Para gestionar estas restricciones con alta eficiencia, se aplican los siguientes principios:

- **Cálculo Direccional Dinámico:** Utiliza operadores ternarios para determinar el vector de avance (`-1` o `+1`) y la fila inicial de origen, unificando la lógica computacional para ambos colores sin duplicación de código.
- **Validación Condicional Anidada:** La evaluación del avance doble está subordinada a la disponibilidad de la primera casilla. Esto evita ciclos de procesamiento innecesarios si la trayectoria está bloqueada.

- **Segregación de Lógica de Desplazamiento:** Separa estructuralmente las validaciones de avance ortogonal (casillas vacías) de las validaciones de captura diagonal (presencia de piezas enemigas).
- **Extensibilidad:** La estructura secuencial facilita la futura integración de reglas excepcionales (Captura al Paso y Coronación) en la etapa final de la rutina de validación.

4.2. Implementación de la Clase `Rook` (Torre)

La clase `Rook` implementa el patrón de piezas de proyección continua (*Sliding Pieces*), calculando trayectorias lineales hasta encontrar un obstáculo topológico.

Para resolver esta mecánica de forma estructurada, la clase aplica los siguientes diseños:

- **Vectores de Dirección Ortogonal:** Emplea una matriz de constantes bidimensionales (`directions[4][2]`) para iterar sobre los cuatro ejes cardinales, consolidando la lógica direccional en un único bloque de control.
- **Proyección Vectorial (Bucle `while`):** Utiliza un ciclo continuo para desplazar la validación casilla por casilla a lo largo del vector activo, actualizando coordenadas secuencialmente.
- **Control Perimetral:** La condición inicial del ciclo verifica los límites de la matriz (`!board.isValidPosition`). Las coordenadas fuera de rango provocan una interrupción del ciclo (`break`), evitando excepciones de acceso a memoria (*Segmentation Faults*).
- **Resolución de Colisiones:** Al detectar una casilla ocupada, el algoritmo evalúa la titularidad de la pieza. Las piezas enemigas se registran como capturas válidas antes de ejecutar una detención incondicional (`break`), cumpliendo la restricción de no superposición.

4.3. Implementación Clase `Bishop` (Alfil)

La clase `Bishop` comparte la arquitectura de proyección continua de la Torre, limitando su influencia a los ejes diagonales.

- **Vectores de Proyección Diagonal:** Define la matriz direccional (`directions[4][2]`) con variaciones simétricas (`1` y `-1`), garantizando que el incremento en el eje de filas sea directamente proporcional al eje de columnas.

- **Reutilización de Lógica de Raycasting:** Mantiene la iteración basada en ciclos `while`, deteniendo la proyección ante colisiones perimetrales (`!isValidPosition`) o intersección con otros objetos instanciados en la matriz.
- **Validación de Capturas:** Emplea el mismo flujo de resolución de colisiones que la Torre, registrando movimientos ofensivos legales al interceptar referencias de bando opuesto.

4.4. Implementación de la Clase `Queen` (Reina / Dama)

La clase `Queen` amalgama las capacidades de la Torre y el Alfil, requiriendo un algoritmo omnidireccional.

- **Fusión Vectorial:** Expande la matriz direccional a 8 vectores (`directions[8][2]`), permitiendo que un iterador único cubra todas las proyecciones ortogonales y diagonales posibles.
- **Homologación del Patrón Deslizante:** Reutiliza la estructura iterativa continua y las validaciones de límite matricial compartidas con el resto de piezas de largo alcance, manteniendo la coherencia arquitectónica.

4.5. Implementación de la Clase `King` (Rey)

La clase `King` pertenece a las piezas de movimiento discreto (*Step Pieces*). Su traslación matemática está limitada a un radio de acción escalar de magnitud 1.

- **Omnidireccionalidad Restringida:** Utiliza la matriz de 8 vectores, pero descarta los ciclos de proyección continua, evaluando únicamente la adyacencia directa.
- **Control de Flujo Discreto (`continue`):** Al calcular la coordenada de destino (`nextPos`), si esta resulta inválida topológicamente, la instrucción `continue` descarta el vector actual y procede a evaluar la siguiente dirección, optimizando el ciclo.
- **Resolución Directa:** La casilla objetivo se evalúa en tiempo $O(1)$. Las posiciones nulas (`nullptr`) se registran como movimientos pasivos y las posiciones enemigas como capturas, sin requerir rutinas de detención por colisión.

4.6. Implementación de la Clase `Knight` (Caballo)

La clase `Knight` implementa un modelo de movimiento no lineal que ignora las restricciones de bloqueo físico de las piezas adyacentes.

- **Vectores de Traslación Asimétrica:** Define la matriz (`directions[8][2]`) combinando incrementos de magnitud 2 y 1, calculando las coordenadas de destino de forma directa sin iterar posiciones intermedias.
- **Exclusión de Lógica de Colisión Intermedia:** Al omitir el raycasting, el algoritmo inherentemente soporta la regla del "salto", interactuando únicamente con el estado de la casilla final.
- **Control de Flujo Perimetral:** Aplica las mismas mitigaciones de error (`continue` ante `!isValidPosition`) y evaluación binaria de casillas destino que la clase `King`.

5. Implementación de la Clase `GameState`

La clase `GameState` gestiona la consistencia lógica de la partida, coordinando el estado espacial (`Board`) con los metadatos transaccionales (turnos, derechos de enroque, condiciones de captura al paso).

Para garantizar transacciones seguras y optimización para la búsqueda en árboles, aplica los siguientes principios:

- **Transacciones Atómicas (`std::stack`):** Almacena metadatos temporales en una estructura `StateInfo` previo a la ejecución de movimientos. Esto posibilita la reversión de estado (`undoState`) en complejidad $O(1)$, manteniendo la integridad de los datos.
- **Observación Segura de Tipos (`dynamic_cast`):** Emplea punteros crudos para consultar instancias sin alterar la propiedad de memoria. Utiliza conversión de tipos dinámica para verificar aserciones polimórficas de forma segura.
- **Gestión de Estados Efímeros:** Inicializa las coordenadas de captura al paso con valores fuera de límites, como `Position(-1, -1)`, creando un mecanismo de invalidación implícito y eficiente.
- **Unificación Lógica (`std::abs`):** Resuelve variaciones asimétricas de movimiento empleando valores absolutos, reduciendo la redundancia de código entre bandos.
- **Inmutabilidad Restringida (`const`):** Protege el encapsulamiento marcando los métodos de consulta (`canCastle`, `getCurrentTurn`) como inmutables.

6. Implementación de la Clase `MoveGenerator`

La clase `MoveGenerator` centraliza la heurística de movimientos, unificando la lógica espacial de las piezas con las validaciones de estado global para retornar conjuntos de jugadas legales.

Para asegurar un alto rendimiento computacional, aplica directrices de optimización en el uso de memoria estándar:

- **Pre-asignación de Memoria (`reserve`):** Reserva proactivamente capacidad en los `std::vector` resultantes, mitigando la sobrecarga por realojos dinámicos durante inserciones continuas.
- **Inserción por Bloques (`insert`):** Aprovecha iteradores estándar (`begin()`, `end()`) para transferir rangos de memoria directamente, mejorando el rendimiento frente a bucles iterativos explícitos.
- **Inmutabilidad del Estado (`const GameState&`):** Opera sobre referencias constantes para evadir la duplicación de objetos pesados durante la generación del árbol.
- **Método `generateLegalMoves`** Filtra los movimientos pseudo-legales aplicando simulaciones temporales (`updateState`). Interpola con `RuleValidator` para descartar estados de *jaque* y emplea la reversión de estado (`undoState`) antes de confirmar el vector de movimientos seguros.

6.1. Reglas Especiales: Enroques (`generateCastlingMoves`)

Calcula la viabilidad de los enroques mediante **validación posicional directa**, consultando las banderas de derechos de `GameState` y confirmando la ausencia de punteros en coordenadas estáticas predefinidas en tiempo $O(1)$.

6.2. Reglas Especiales: Captura al Paso (`generateEnPassantMoves`)

Evalúa la ventana transaccional del movimiento verificando:

- Descarte algorítmico inmediato si la coordenada objetivo es inválida.
- Cálculo relativo de índices de fila para escanear adyacencias ortogonales.
- Validación estricta de herencia usando `dynamic_cast` para garantizar que las piezas atacantes cumplan con las reglas de clase.

7. Implementación de la Clase `RuleValidator`

La clase `RuleValidator` opera bajo el Principio de Responsabilidad Única (SRP), limitándose a evaluar la exposición del Rey ante amenazas del bando opuesto. La clase se define como `static` para omitir la sobrecarga de instanciación recurrente.

7.1. Algoritmo de Escaneo Inverso

En lugar de procesar exhaustivamente el árbol de movimientos enemigos, el validador aplica un **escaneo desde la posición objetivo**. Utilizando la coordenada del Rey como punto de origen, proyecta vectores hacia el perímetro para identificar colisiones de forma deductiva.

7.2. Mecanismos de Detección

La evaluación se fundamenta en dos aproximaciones geométricas:

- **Escaneo Expansivo (*Raycasting*):** Evalúa las amenazas de piezas deslizantes. Proyecta líneas iterativas; si intercepta una pieza aliada, asume bloqueo y aborta la línea. Si intercepta una amenaza enemiga válida en ese vector, la aserción es positiva.
- **Escaneo Puntual:** Evalúa amenazas de alcance discreto (Caballos, Peones). Transforma coordenadas mediante offsets precalculados y verifica los punteros de destino en tiempo $O(1)$.

7.3. Evaluación Perezosa (*Early Exit*)

El algoritmo finaliza su ejecución al identificar la primera aserción verdadera, omitiendo el cálculo del resto de la matriz. Esta optimización es crítica para reducir el consumo de ciclos de reloj durante operaciones recursivas.

8. Implementación de la Clase `Player`

Esta clase abstracta define el contrato base para entidades con capacidad de decisión. Implementa el Principio de Inversión de Dependencias (DIP) aislando la arquitectura central del origen de los comandos de juego, ya sea entrada estándar de usuario u operaciones heurísticas.

- **Estado Inmutable:** Gestiona el identificador de bando (`Color`) asignado en tiempo de instanciación, el cual se blinda tras métodos de acceso de solo lectura.

- **Delegación de Flujo (`getMove`):** Obliga la implementación del método virtual puro en subclases, trasladando la complejidad de la lógica de selección de jugadas fuera del ciclo principal de `Game`.

8.1. Implementación de la Clase `HumanPlayer`

Representa una implementación concreta para el manejo de I/O por consola. Funciona como una capa de sanitización y validación entre la entrada del usuario y la lógica de dominio.

- **Transformación Léxica (`parsePosition`):** Convierte entradas de notación algebraica (ej. ."e2") en índices matriciales $(0 - 7)$ en tiempo $O(1)$, utilizando aritmética de caracteres ASCII y ajustando el eje coordenado a la matriz interna.
- **Validación de Estado:** El método `getMove` gestiona los errores de flujo (*buffer streams*) e implementa un bucle iterativo que coteja el formato procesado contra el conjunto legal proporcionado por el generador, previniendo excepciones y corrupción de estado de la partida.

9. Implementación de la Clase `Game`

Representa la **fachada principal** de la lógica del ajedrez (Capa de Dominio) y opera como una máquina de estados. Diseñada bajo el Principio de Responsabilidad Única (SRP), delega el manejo de la interfaz a los controladores visuales (como `ChessController`). Su rol fundamental es servir de frontera arquitectónica, garantizando que el entorno gráfico consuma las reglas del ajedrez sin acoplarse a su gestión de memoria subyacente.

- **Encapsulación y Despacho Dinámico:** Mantiene localmente el `GameState` y el `MoveGenerator` para calcular el árbol de jugadas legales, delegando la elección mediante polimorfismo inyectado (`Player*`) sin importar si el turno pertenece a un humano o a un algoritmo.
- **Desacoplamiento Estructural (DTOs):** Puesto que la clase `Move` gestiona punteros dinámicos (`unique_ptr`), `Game` actúa como intermediario estructural. Exporta a los controladores únicamente coordenadas primitivas (`std::vector<Position>`), previniendo fugas de memoria en la capa de interfaz.

- **Procesamiento Transaccional (`processMove`):** Adapta el motor al flujo asíncrono de la GUI. Recibe coordenadas puras, las valida contra el árbol de jugadas, procesa operaciones complejas (ej. coronaciones) y auto-evalúa el fin de partida (Jaque Mate o Ahogado), limpiando simultáneamente el historial de estados futuros (`redoHistory`).

- **Consultas Inmutables (*Queries*):** Provee a los controladores una vista constante del tablero (`getBoard`) y métodos reactivos (`isInCheck`, `isGameOver`) para que la interfaz reaccione a eventos lógicos sin permisos de mutación sobre el estado principal.

- **Detección Geométrica de Movimientos Especiales:** Incorpora métodos de evaluación (`isKingsideCastle`, `isQueensideCastle`, `isEnPassant`, `isPromotionMove`) que analizan las coordenadas de origen y destino para confirmar la ejecución de reglas excepcionales. Esto suministra información precisa al sistema de registro de movimientos (logger) sin requerir re-cálculos lógicos.

- **Gestión de Reversión Condicionada (Undo):** Implementa el método `undoLastMove` apoyado por un arreglo de control de estado (`hasUsedUndo`), el cual restringe la reversión a una única ejecución por ciclo de turno por jugador. Bloquea la acción si la partida concluyó por Jaque Mate y gestiona la restauración íntegra del estado anterior del tablero.

- **Navegación de Análisis Temporal:** Introduce los métodos `stepBackwardAnalysis` y `stepForwardAnalysis`, los cuales trasladan secuencialmente los objetos `Move` entre las estructuras `moveHistory` y `redoHistory`. Esta operación manipula el estado del tablero omitiendo la validación activa de reglas y las condiciones de fin de juego, optimizando el rendimiento computacional durante la revisión visual de la partida.

- **Compatibilidad Iterativa (`playTurn`):** Preserva un bucle lógico secuencial puro, permitiendo la ejecución de partidas automatizadas o pruebas de estrés en consola mediante una única llamada cíclica.